

Referencia de módulos

Descripción completa de la superficie pública de cada módulo del sistema. Toda la información está derivada directamente del código fuente.

ingesta.py — Carga y limpieza de datos

leer_csv(ruta_archivo)

Lee un CSV y retorna una lista de registros. **Retorna:** `list[dict]` — cada dict tiene claves `id` (str), `texto` (str), `fecha` (str) y `usuario` (str; cadena vacía si la columna no existe). **Excepciones:** `FileNotFoundError`, `ValueError` (faltan columnas `id`, `texto`, `fecha`). Intenta UTF-8 y, si falla con `UnicodeDecodeError`, reintenta en latin-1.

limpiar_texto(texto)

Normaliza texto para el análisis. **Retorna:** `str` — en minúsculas, sin URLs ni @menciones, conservando la palabra de cada hashtag, letras acentuadas del español, ñ, dígitos y espacios, con espacios múltiples colapsados.

detectar_duplicados(datos)

Separa registros únicos de duplicados basándose en el texto limpio. **Retorna:** `tuple[list[dict], list[dict]]` — `datos_unicos` (incluye clave `texto_limpio`) y `duplicados`.

procesar_datos(ruta_archivo)

Pipeline completo: lectura → limpieza → deduplicación → filtro de ruido. **Retorna:** `tuple[list[dict], dict]` con estadísticas (`total_original`, `total_unicos`, `total_duplicados`, `duplicados_ids`, `total_ruido`).

clasificador.py — Análisis de sentimiento

AnalizadorSentimiento

Analiza el sentimiento de textos en español mediante un léxico ponderado de **375 entradas**, que cubre vocabulario formal, coloquialismos y jerga peruana, además de intensificadores. No utiliza modelos preentrenados de IA: la clasificación es puramente léxica con la biblioteca estándar de Python. Hereda de `AnalizadorBase` y sobrescribe `clasificar()` (polimorfismo).

```
AnalizadorSentimiento(umbral_muy_positivo=2.0, umbral_muy_negativo=-2.0)
```

Escala del léxico

Peso	Significado	Ejemplos
+3	Muy positivo	maravilloso, excelente, espectacular
+2	Positivo	bueno, mejora, recomiendo
+1	Levemente positivo	aceptable, normal, razonable
-1	Levemente negativo	preocupante, retraso, deficiente
-2	Negativo	malo, denuncia, negligencia, vergüenza
-3	Muy negativo	fraude, escándalo, corrupto, jerga ofensiva
+0.3 a +1.0	Intensificador	muy (0.5), absolutamente (1.0), siempre (0.3)

Negadores: `no`, `nunca`, `jamás` / `jamás`, `ni`, `tampoco` y `sin` invierten el signo del peso de la palabra siguiente; la pareja consume ambos tokens. Si el negador es la última palabra, no suma peso.

clasificar(texto_limpio)

Retorna: `dict` con `categoria`, `puntuacion` (redondeada a 4 decimales), `puntuacion_raw` y `palabras_clave`. Fórmula: `puntuacion = round((puntuacion_raw / max(n_palabras, 1)) * 10, 4)`.

Puntuación normalizada	Categoría
≥ 2.0	muy positivo

<code>> 0 y < 2.0</code>	positivo
<code>= 0</code>	neutro
<code>≥ -2.0 y < 0</code>	negativo
<code>< -2.0</code>	muy negativo

Otros métodos: `analizar_lote(datos)`, `filtrar_negativos(datos, umbral=-1.0)`, `obtener_estadisticas(datos)`, `obtener_lexicon()` (copia de solo lectura, 375 entradas) y `obtener_umbrales()`.

alertas.py — Gestión de alertas

```
GestorAlertas(umbral=-2.0, porcentaje_crisis=30.0)
```

`verificar_comentario(comentario)` retorna un dict de alerta si `puntuacion < umbral`, o `None`.
`verificar_crisis_global(estadisticas, total_comentarios)` retorna una alerta global (`tipo = "crisis_global"`, `id_comentario = "GLOBAL"`) si el % de negativos supera el umbral de crisis. `procesar_lote(datos)` genera alertas individuales y las imprime. Además: `obtener_alertas()`, `total_alertas()`, `obtener_umbral()`, `configurar_umbral(nuevo)`.

persistencia.py — Almacenamiento multi-formato MYSQL

GestorPersistencia

```
GestorPersistencia(directorio_salida="datos_salida", base_datos="crisis_monitor")
```

El constructor crea el directorio de salida si no existe, crea la base de datos MySQL `crisis_monitor` (con `CREATE DATABASE IF NOT EXISTS ... CHARACTER SET utf8mb4`) si no existe e inicializa las tablas `comentarios` y `alertas`. La conexión usa los valores de `CONFIG_MYSQL` (host, port, user, password) definidos al inicio del módulo. **Excepciones:** `RuntimeError` si MySQL no puede inicializarse.

El esquema (DDL de MySQL) está **embebido** en la clase. El archivo `schema.sql` es opcional, para crear las tablas manualmente con `mysql -u root -p < schema.sql`; el sistema **no** lo lee automáticamente en el arranque.

Archivos planos

`guardar_txt(datos, nombre_archivo="comentarios.txt")` exporta todos los comentarios en texto plano.
`guardar_csv_filtrado(datos, categorias_filtro=None, nombre_archivo="negativos.csv")` exporta en CSV solo las categorías indicadas (por defecto `["negativo", "muy negativo"]`). `guardar_pickle` / `cargar_pickle` serializan y deserializan cualquier objeto Python.

Operaciones DML (MySQL)

Método	Verbo DML	Retorna
<code>insertar_comentario(comentario)</code>	INSERT individual	int — id AUTO_INCREMENT
<code>insertar_lote_comentarios(datos)</code>	INSERT masivo (<code>executemany</code>)	int — filas insertadas
<code>insertar_alerta(alerta)</code>	INSERT en <code>alertas</code>	int — id AUTO_INCREMENT
<code>seleccionar_comentarios(categoria=None, limite=None)</code>	SELECT (orden <code>fecha_proceso</code> DESC)	list[dict]
<code>seleccionar_alertas(solo_activas=False)</code>	SELECT de <code>alertas</code>	list[dict]
<code>actualizar_categoria(id_original, nueva_categoria)</code>	UPDATE por ID original	int — filas afectadas
<code>resolver_alerta(id_alerta)</code>	UPDATE (<code>activa = 0</code>)	int — filas actualizadas
<code>eliminar_comentario(id_original)</code>	DELETE por ID original	int — filas eliminadas
<code>limpiar_alertas_resueltas()</code>	DELETE (<code>activa = 0</code>)	int — filas eliminadas

Internamente las consultas usan marcadores de parámetro de MySQL (`%s`) y cursores `dictionary=True` para los SELECT. Los errores de MySQL (`mysql.connector.Error`) se envuelven en `RuntimeError`.

guardar_lote_completo(datos, alertas) → dict

Persiste datos y alertas en todos los formatos en una sola llamada. **Retorna:** dict con `txt`, `csv`, `pickle`,

`mysql_comentarios` (int) y `mysql_alertas` (int).

reportes.py — Generación de reportes

`generar_reporte_crisis(datos, alertas, nombre_archivo="reporte_crisis.txt")` genera el reporte completo: (1) encabezado con métricas globales, (2) distribución de categorías con barras ASCII, (3) top 5 palabras negativas, (4) resumen de alertas, (5) top 5 comentarios más negativos y usuarios con más negativos.

% de negativos	Nivel
≥ 60%	CRÍTICO
≥ 30%	ALTO
≥ 10%	MODERADO
< 10%	NORMAL

Esquema de la base de datos MYSQL

El esquema (DDL de MySQL) está embebido en `GestorPersistencia`. La base de datos `crisis_monitor` (MySQL, charset `utf8mb4`) se compone de dos tablas con sus índices. También se incluye, como conveniencia opcional, un archivo `schema.sql` equivalente.

Tabla comentarios

Columna	Tipo	Restricción
id	INT	PK, AUTO_INCREMENT
id_original	VARCHAR(64)	NOT NULL
texto_original	TEXT	NOT NULL
texto_limpio	TEXT	NOT NULL
usuario	VARCHAR(120)	NOT NULL, DEFAULT ""
fecha	VARCHAR(40)	NOT NULL
categoria	VARCHAR(20)	NOT NULL
puntuacion	FLOAT	NOT NULL
fecha_proceso	VARCHAR(20)	NOT NULL

Índices: `idx_cat` (categoria), `idx_fecha` (fecha_proceso).

Tabla alertas

Columna	Tipo	Restricción
id	INT	PK, AUTO_INCREMENT
id_comentario	VARCHAR(64)	NOT NULL (o "GLOBAL")
texto	TEXT	NOT NULL
usuario	VARCHAR(120)	NOT NULL, DEFAULT ""
puntuacion	FLOAT	NOT NULL
umbral	FLOAT	NOT NULL
fecha_alerta	VARCHAR(20)	NOT NULL
activa	TINYINT	NOT NULL, DEFAULT 1

Índice: `idx_act` (activa).

Opciones del menú principal

Opción	Función	Datos requeridos
1	Cargar y analizar CSV	Ninguno
2	Ver estadísticas del análisis actual	CSV cargado en sesión

3	Consultar base de datos	Base de datos existente
4	Gestionar alertas	Base de datos existente
5	Generar reporte de crisis	CSV cargado en sesión
6	Exportar datos	CSV cargado en sesión
7	Demo DML (INSERT/SELECT/UPDATE/DELETE)	CSV cargado en sesión
8	Configurar umbral de alertas	Ninguno
9	Crear CSV de ejemplo	Ninguno
0	Salir	Ninguno

Véase también: [Tutorial de inicio](#) · [Guías de tareas comunes](#) · [Arquitectura del sistema](#).